

Risque de l'AI générative sur la qualité de logiciel et sa maintenance

L'AI générative a connu une avancée ces dernières années, surtout dans le domaine de développement informatique, où elle est devenue un outil essentiel pour les développeurs même pour les étudiants en phase d'apprentissage. Elle les aide à coder, s'informer et même à rédiger des rapports ou de la documentation.

Cependant, l'utilisation abusive et irresponsable de cet outil au niveau du développement d'application peut engendrer des risques pour la qualité de logiciel et sa maintenance, sujets que nous allons évoquer dans ce rapport.

1. Risque pour la qualité de logiciel

La qualité d'un logiciel ne se limite pas au fait qu'un programme fonctionne. Un logiciel de haute qualité est un système qui présente une architecture robuste, sécurisée et scalable, capable d'évoluer dans le temps et facile à maintenir.

C'est là où l'AI générative montre ses limites, et l'on peut citer plusieurs points :

- Mauvaise architecture : La phase la plus critique dans la création d'un logiciel est la conception architecturale. L'objectif est de concevoir une application capable de séparer les responsabilités, pour but de créer un pattern lisible et facile à maintenir. Malheureusement, l'AI n'est pas un humain, elle n'a pas la vision du projet. Elle propose une solution qui fonctionne mais sans respecter une architecture saine et sans anticiper l'évolution future de l'application.

- Failles de sécurité : Pour ce point précis, l'IA fait des ravages même avec les modèles les plus avancés, le problème de sécurité persiste. Pour les failles classiques que n'importe quel développeur débutant saurait éviter, comme l'injection SQL, l'AI au contraire les insère partout dans le code. C'est dû au fait que les IA sont entraînées sur du code public qui

contient des vulnérabilités, l'IA propose alors des scripts qui rendent notre logiciel vulnérable à n'importe quelle attaque.

- **Rend le développeur faible techniquement :** Pour les développeurs expérimentés, cette inquiétude ne les concerne pas trop, le grand souci concerne les développeurs débutants et les étudiants en phase d'apprentissage. La génération facile de code ou de rapports rend ces derniers trop dépendants de l'IA, au point qu'ils ne peuvent plus réaliser de simples tâches. Cela risque de créer de futurs ingénieurs incapables de concevoir des applications ou même d'écrire une ligne de code sans l'aide des AI. C'est la société qui en paiera le prix, car l'IA peut devenir inaccessible n'importe quand.

- **Le débogage :** Dans un processus de développement logiciel, il y a forcément des erreurs qui apparaissent dans la console et le but est de les résoudre pour avancer dans le projet. Le débogage est réalisable si c'est nous qui avons codé le logiciel. En revanche, si nous avons laissé l'AI générer tout le code, cela devient très difficile, car nous ne maîtrisons pas les liens entre les fichiers, les bibliothèques et les fonctions générées. Même si l'on essaie d'utiliser la même IA pour déboguer, cela finit souvent par créer beaucoup d'autres bugs.

2. La Maintenance à long terme

La maintenance représente la phase la plus longue, importante et coûteuse du cycle de vie d'un logiciel. L'IA peut fragiliser cette étape.

- **Intervention complexe :** Si notre logiciel de base est codé simplement en collant des scripts générés par IA, on serait incapable d'intervenir rapidement lors d'un bug de production. Étant donné que le projet n'a pas été conçu par nous-mêmes, nous n'avons aucun repère pour identifier l'origine du problème.

- **Refactorisation** : Le code généré est souvent rempli de redondances inutilement complexes, ce qui nous donne des fichiers de plusieurs centaines de lignes. Plus il y a de code inutile et redondant, plus il est difficile de le mettre à jour sans créer des bugs.

3. Recommandations

Pour éviter ces risques tout en utilisant l'IA, le meilleur moyen est de l'utiliser uniquement comme un outil d'assistance pas un outil de conception d'une application de A à Z. Il faut que la conception générale et la logique du code soient écrites par nous-mêmes.

Conclusion

L'IA est un outil puissant, mais ne doit jamais remplacer un vrai développeur. La qualité et la maintenabilité dépendent de la capacité des développeurs à rester critiques envers eux-mêmes, ainsi qu'envers les suggestions de l'IA.

SOURCES :

https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality

<https://cacm.acm.org/research-highlights/asleep-at-the-keyboard-assessing-the-security-of-github-copilots-code-contributions/>

<https://www.ijfmr.com/papers/2025/6/59519.pdf>

<https://snyk.io/fr/blog/ai-code-generation-code-security-quality-benefits-risks-top-tools/>

AISSANI YANNEL

BUT3 INFO

2025/2026